

Notificaciones

Concepto

Una notificación en Laravel es una clase que permite enviar información a los usuarios a través de varios canales, como correo electrónico, SMS, Slack, o incluso guardar la notificación en la base de datos para mostrarla en la interfaz de usuario. Estos elementos nos permiten enviar distintos tipos de información, como pueden ser: recordatorios de restablecimiento de contraseña, una alerta de nuevo comentario, o una confirmación de pedido.

Elementos

Toda notificación consta de los siguientes elementos:

- **Notificación:** clase que define las características de la notificación. Esta clase es altamente configurable para satisfacer varios escenarios de ser necesario.
- **Canal (*vía*):** indica cual será el medio por el cual se enviará la notificación.
- **Datos:** los valores que serán enviados mediante la notificación, dado que la misma puede incluir distintos tipos de información.
- **Usuario:** el usuario que recibirá la información. Un sistema que precise el uso de notificaciones debe contar previamente con un mecanismo de autenticación implementado.
- **Evento (*opcional*):** el evento que desencadena el proceso de notificación.

¿Qué es un evento?

Un **evento** en Laravel es una acción que ocurre en tu aplicación y a la que puedes "suscribirte" para ejecutar una o más tareas en respuesta a esa acción. Los eventos utilizan la arquitectura "**pub/sub**": un elemento es el *publicador* (en este caso, el evento) y otro elemento es el *suscriptor* que se "subcribe" para obtener información del evento y realizar una acción cuando el mismo ocurre. Los suscriptores se conocen como **listeners** o escuchadores.

La principal ventaja de los eventos es que **desacoplan** diferentes partes de tu código. En lugar de que la lógica de tu aplicación dependa directamente de otras clases, un componente simplemente dispara un evento, y cualquier otro componente interesado en esa acción puede responder.

Tomemos el siguiente ejemplo: supongamos que cada vez que un usuario se registra en la aplicación, queremos que ocurran las siguientes tareas en respuesta a dicho evento:

1. Se envíe un email al correo del nuevo usuario dándole la bienvenida al sistema y mostrándole tips útiles.
2. Se envíe una notificación web a cada usuario administrador del sistema, indicando que un nuevo usuario se ha registrado.

3. Se programe el envío de otro email para que dentro de una semana se envíe automáticamente al nuevo usuario, mostrándole más información de uso de la plataforma junto con una encuesta de opinión.

En lugar de poner todo este código en el controlador, podríamos crear un nuevo evento llamado “**UsuarioRegistradoEvent**” y luego una serie de **listeners** que “escuchen” cuando ocurre el evento y ejecuten su lógica.

Como este es un tema muy extenso, dejamos el enlace de la documentación oficial en caso de querer seguir profundizando: <https://laravel.com/docs/12.x/events>

Rendimiento

Antes de avanzar es necesario mencionar un detalle importante. Las notificaciones se ejecutan de forma **sincrónica**, es decir que al enviar una notificación el programa debe esperar a que finalice el proceso para continuar con las siguientes tareas. Esto puede causar que la aplicación funcione más lentamente que antes, especialmente si se van a enviar varias notificaciones al mismo tiempo.

Para evitar esto se debería programar un proceso de trabajos en paralelo (**Queue Workers**) y trabajar con los eventos del sistema. Esto puede configurarse en el mismo servidor que estamos usando para nuestra aplicación o utilizar un servidor externo dedicado: Redis, AmazonSQS, Beanstalkd u otro servicio.

Funcionamiento

Crear una notificación

Ejemplo

Vamos a crear una notificación sencilla: cada vez que se cree una transacción, mostraremos una notificación web al usuario que la haya creado. Si bien no es un caso extremo, nos sirve para ver cada uno de los pasos necesarios para agregar una notificación a nuestro sistema.

Utilizaremos por defecto el canal de la base de datos, ya que es el más sencillo de implementar

Veamos cada uno de los pasos necesarios para implementarlo.

1) Crear la tabla de notificaciones

Debido al canal elegido, necesitamos la tabla de notificaciones de Laravel. Crear su migración es tan sencillo como ejecutar uno de los dos comandos siguientes (ambos son análogos):

```
php artisan make:notifications-table
```

```
php artisan notifications:table
```

Una vez creado el archivo, podemos ejecutar dicha migración para crear la tabla en nuestra base de datos.

2) Especificar el modelo usado para las notificaciones

En Laravel podemos enviar notificaciones a cualquier modelo, pero por lo general queremos enviar notificaciones a los usuarios. Así que abrimos el modelo User y le agregaremos el trait Notifiable.

```
use Illuminate\Notifications\Notifiable;

class User extends Authenticatable
{
    use Notifiable;
}
```

3) Crear la notificación

Utilizamos el siguiente comando, el cual creará una nueva clase en la carpeta **app/Notifications**:

```
php artisan make:notification TransaccionCreada
```

4) Configurar la clase

Editaremos la clase para nuestro caso particular. Primero, agregaremos un nuevo atributo a la clase para obtener información de la transacción creada y luego la inicializaremos en el constructor. De esta forma podemos pasar información externa que queremos mostrar en una notificación.

```
protected $transaccion;

public function __construct($transaccion)
{
    $this->transaccion = $transaccion;
}
```

Luego modificamos el método **via()** para indicar que las notificaciones serán guardadas en la base de datos:

```
public function via(object $notifiable): array
{
    return ['database'];
}
```

```
}
```

Por último, modificamos el método **toArray()**. Este método necesita como mínimo el valor **"message"** para saber cual es el mensaje que se enviará mediante la notificación. Como extra, podemos agregar otros datos si lo consideramos oportuno para nuestra notificación:

```
return [  
    'message' => "Se ha creado una transaccion por \${$this->transaccion->monto}",  
    'transaccion_id' => $this->transaccion->id,  
];
```

5) Enviar la notificación

Para enviar la notificación podemos hacerlo de dos formas diferentes. La primera aproximación consiste en utilizar el método **notify()** sobre uno de los usuarios del sistema; como requisito previo debemos implementar el trait **Notifiable** (ver paso 2)

```
$usuario = auth()->user();  
$usuario->notify(new CursoEditado());
```

Como segunda alternativa podemos usar el método **send()** de la clase Notification. Si elegimos esta opción podemos enviar la notificación a multiples usuarios, y no es necesario implementar el trait Notifiable previamente:

```
use Illuminate\Support\Facades\Notification;  
$usuario = auth()->user();  
Notification::send($usuario, new CursoEditado());
```

6) Mostrar la notificación

Independientemente de la implementación que elijamos para mostrar los datos en el frontend de la aplicación, podemos acceder a los datos de la notificación accediendo a la relación **"notifications"** (creada automáticamente al implementar el trait Notifiable en el paso 2). También contamos con dos variantes de esta relación:

- Si queremos acceder a las notificaciones pendientes, podemos usar **unreadNotifications**
- Si queremos acceder a las notificaciones leídas previamente podemos usar **readNotifications**

```
// Podemos reemplazar notifications por unreadNotifications o
readNotifications

foreach ($usuario->notifications as $notificacion) {
    echo $notification->data;
}
```

7) Marcar una notificación como leída

Contamos con distintas estrategias. Primero, podemos iterar sobre un grupo de notificaciones y usar el método **markAsRead()** para marcarlas como leídas:

```
$user = App\Models\User::find(1);
foreach ($user->unreadNotifications as $notification) {
    $notification->markAsRead();
}
```

Otra opción es utilizar el mismo método sobre un grupo de notificaciones:

```
$user->unreadNotifications->markAsRead();
```

También podemos usar una consulta para actualizar la fecha de lectura por la fecha actual, causando el mismo efecto:

```
$user->unreadNotifications()->update(['read_at' => now()]);
```

La última alternativa es eliminar directamente las notificaciones deseadas:

```
$user->notifications()->delete();
```

Para más información contamos con la documentación oficial sobre notificaciones:

<https://laravel.com/docs/12.x/notifications>